

Gereksinim Analizi Raporu

1. Belge bilgileri

Hedef Platform: Web Uygulaması

Backend: ASP.NET Core Web API

Frontend: Angular

Veri Tabanı: SQL Server

2. Projenin amacı

Projenin amacı, kullanıcıların günlük gelir ve giderlerini güvenli biçimde kaydedebildiği; kayıtlarını kategori ve tarih bazında inceleyebildiği; toplam gelir, toplam gider ve kalan bakiyesini görebildiği bir web uygulaması geliştirmektir.

Uygulama küçük ve yönetilebilir parçalar halinde geliştirilecek; çalışan backend, Angular arayüzü ve kalıcı SQL Server veritabanıyla tamamlanacaktır.

3. Kapsam

3.1 Zorunlu kapsam

- Gelir ve gider kayıtlarını ekleme, görüntüleme, güncelleme ve silme.
- Gelir ve gider kategorilerini yönetme.
- İşlemleri tarih aralığına ve kategoriye göre filtreleme.
- Toplam gelir, toplam gider ve kalan bakiye hesaplama.
- Formlarda doğrulama yapma ve anlaşılır hata mesajları gösterme.
- Verileri SQL Server üzerinde kalıcı olarak saklama.
- API işlemlerini Swagger veya Postman ile test edebilme.

3.2 Projeye dahil edilen opsiyonel kapsam

- Kullanıcı kayıt ve giriş ekranları.
- JWT access token ile kimlik doğrulama.
- Refresh token ile güvenli oturum yenileme.
- Aylık gelir-gider grafiği.
- İşlem listesinde sayfalama.

3.3 Kapsam dışı özellikler

- Banka hesaplarıyla otomatik veri alışverişi.
- Birden fazla para birimi ve kur dönüşümü.
- E-posta doğrulama ve şifre sıfırlama.
- Yönetici paneli ve kullanıcı yönetimi.
- Dosya, fiş veya fatura yükleme.
- Bildirim ve bütçe limiti özellikleri.

4. Kullanıcı ve temel senaryo

Sistemin temel aktörü kayıtlı kullanıcıdır. Kullanıcı yalnızca kendi kategorilerine, işlemlerine ve oturum bilgilerine erişebilir.

1. Kullanıcı e-posta adresi ve parolasıyla kayıt olur.
2. Giriş yaptıktan sonra JWT ile korunan uygulama ekranlarına erişir.
3. Gelir veya gider kategorisi oluşturur.
4. Kategori, tür, tutar, tarih ve açıklama bilgileriyle işlem ekler.
5. İşlemleri listeler; tarih veya kategori filtresi uygular.
6. Gerekirse işlemi günceller ya da siler.
7. Ana ekranda toplam gelir, toplam gider ve kalan bakiyeyi görür.
8. Aylık grafikten gelir-gider değişimini inceler.

5. İşlevsel gereksinimler

5.1 Kimlik doğrulama

Kullanıcı ad, soyad, benzersiz e-posta ve parola ile kayıt olabilmelidir. Kullanıcı e-posta ve parola ile giriş yapabilmelidir. Başarılı girişte kısa ömürlü JWT access token ve refresh token üretilmelidir. Access token süresi dolduğunda geçerli refresh token ile yeni token çifti alınabilmelidir. Kullanıcı çıkış yaptığında ilgili refresh token iptal edilmelidir. Hatalı veya süresi dolmuş token ile korunan kaynaklara erişim engellenmelidir.

5.2 Kategori yönetimi

Kullanıcı kendi kategorilerini listeleyebilmelidir. Kullanıcı gelir veya gider türünde yeni kategori ekleyebilmelidir. Kullanıcı kendi kategorisinin adını ve durumunu güncelleyebilmelidir. Kullanıcı artık kullanılmayacak kategoriyi pasifleştirebilmelidir. İşlem formunda yalnızca seçilen işlem türüyle uyumlu aktif kategoriler gösterilmelidir.

5.3 Gelir-gider işlemleri

Kullanıcı gelir veya gider kaydı ekleyebilmelidir. Kullanıcı kendi işlemlerini listeleyebilmelidir. Kullanıcı tek bir işlem kaydının detayını görüntüleyebilmelidir. Kullanıcı kendi işlem kaydını güncelleyebilmelidir. Kullanıcı kendi işlem kaydını silebilmelidir. İşlemler başlangıç ve bitiş tarihine göre filtrelenebilmelidir. İşlemler kategoriye ve işlem türüne göre filtrelenebilmelidir. Filtreler birlikte kullanılabilir. İşlem listesi tarih sırasına göre sayfalanabilmelidir.

5.4 Özet ve raporlama

Kullanıcının toplam geliri hesaplanmalıdır.

Kullanıcının toplam gideri hesaplanmalıdır.

Bakiye, `toplam gelir-toplam gider` formülüyle hesaplanmalıdır.

Özet değerleri seçilen tarih aralığına göre hesaplanabilmelidir.

Gelir ve giderler aylara göre gruplanarak grafikte gösterilmelidir.

6. İş kuralları

İşlem türü yalnızca `Income` veya `Expense` olabilir.

- İşlem tutarı sıfırdan büyük olmalıdır; giderler negatif tutarlı olamaz
- Seçilen kategori işlemle aynı türe ve aynı kullanıcıya ait olmalıdır.
- Aynı kullanıcı aynı türde aynı kategori adını birden fazla kez oluşturamamalıdır.
- Pasif kategori yeni işlemlerde kullanılamamalı, geçmiş işlemlerde korunmalıdır.
- Kullanıcı başka bir kullanıcıya ait kategori, işlem veya refresh token kaydına erişememelidir.
- E-posta adresi sistem genelinde benzersiz olmalıdır.
- Parolalar açık metin olarak kaydedilmemeli, güçlü bir parola hash algoritması kullanılmalıdır.
- Ham refresh token veritabanında saklanmamalı; yalnızca token özeti saklanmalıdır.
- Başlangıç tarihi bitiş tarihinden sonra olamaz.
- Sayfa numarası ve sayfa boyutu pozitif olmalıdır; sayfa boyutuna üst sınır uygulanmalıdır.
- Kayıt oluşturma zamanları UTC olarak saklanmalıdır.

7. Veri gereksinimleri

`Users` Kullanıcı hesabı ve parola hash bilgisini saklamak

`Categories` Kullanıcıya ait gelir-gider kategorilerini saklamak

`Transactions` Gelir ve gider kayıtlarını tek tabloda saklamak

`RefreshTokens` JWT oturum yenileme ve token iptal geçmişini saklamak

8. API gereksinimleri

Planlanan temel endpoint'ler:

POST	<code>/api/auth/register</code>	Yeni kullanıcı oluşturur
POST	<code>/api/auth/login</code>	Kullanıcı girişi yapar
POST	<code>/api/auth/refresh</code>	Token çiftini yeniler
POST	<code>/api/auth/logout</code>	Refresh tokenı iptal eder
GET	<code>/api/categories</code>	Kullanıcının kategorilerini listeler
POST	<code>/api/categories</code>	Yeni kategori oluşturur
PUT	<code>/api/categories/{id}</code>	Kategoriye günceller
DELETE	<code>/api/categories/{id}</code>	Kategoriye pasifleştirir
GET	<code>/api/transactions</code>	İşlemleri filtreli ve sayfalı listeler

GET	/api/transactions/{id}	İşlem detayını getirir
POST	/api/transactions	Yeni gelir veya gider ekler
PUT	/api/transactions/{id}	İşlemi günceller
DELETE	/api/transactions/{id}	İşlemi siler
GET	/api/dashboard/summary	Gelir, gider ve bakiye özetini getirir
GET	/api/dashboard/monthly-chart	Aylık grafik verisini getirir

9. Angular ekran gereksinimleri

Kayıt	Ad, soyad, e-posta, parola ve parola tekrarı formu
Giriş	E-posta ve parola formu
Ana Sayfa / Özet	Toplam gelir, toplam gider, bakiye ve aylık grafik
İşlem Listesi	Sayfalı tablo, tarih, kategori ve tür filtreleri
Yeni İşlem	Tür, kategori, tutar, tarih ve açıklama formu
İşlem Güncelleme	Seçilen işlemin bilgileriyle doldurulmuş düzenleme formu
Kategori Yönetimi	Kategori listeleme, ekleme, düzenleme ve pasifleştirme

10. Doğrulama ve hata mesajları

- Zorunlu alanlar hem Angular formunda hem API tarafında doğrulanmalıdır.
- E-posta biçimi ve benzersizliği kontrol edilmelidir.
- Parola için minimum güvenlik kuralları uygulanmalıdır.
- Tutar sıfırdan büyük olmalıdır.
- İşlem tarihi ve tarih aralığı geçerli olmalıdır.
- Kategori kullanıcıya ait, aktif ve işlem türüyle uyumlu olmalıdır.
- Hatalar standart bir API hata modeliyle ve kullanıcıya anlaşılır Türkçe mesajlarla dönmelidir.
- Kaynak bulunmadığında **404** , doğrulama hatasında **400** , kimlik doğrulama hatasında **401** , yetkisiz kaynak erişiminde **403** kullanılmalıdır.

11. İşlevsel olmayan gereksinimler

Güvenlik

- Tüm kullanıcı verileri kullanıcı kimliğine göre izole edilmelidir.
- Parolalar ve tokenlar loglara yazılmamalıdır.
- Bağlantı bilgileri ve JWT anahtarı kaynak koda eklenmemelidir.
- Korunan API endpoint'leri JWT doğrulaması yapılmalıdır.

Performans

- İşlem listesi veritabanı seviyesinde sayfalanmalıdır.
- Tarih, kategori, kullanıcı ve tür filtrelerinde uygun indeksler kullanılmalıdır.

- Dashboard toplamaları belleğe tüm kayıtlar alınmadan SQL sorgusuyla hesaplanmalıdır.

Kullanılabilirlik

- Form hata mesajları ilgili alanın yanında gösterilmelidir.
- Gelir, gider ve bakiye değerleri tutarlı renk ve para biçiminde sunulmalıdır.
- Boş liste, yükleniyor ve hata durumları kullanıcıya açıkça gösterilmelidir.

Bakım yapılabilirlik

- Sınıf, metot ve değişken adları anlamlı olmalıdır.
- Controller içinde uzun iş kuralları tutulmamalıdır.
- API giriş ve çıkışlarında entity yerine DTO kullanılması tercih edilmelidir.
- Tekrarlanan kod ortak bileşenlere ayrılmalıdır.

12. Kabul kriterleri

Proje aşağıdaki senaryolar hatasız tamamlandığında kabul edilebilir:

1. Yeni kullanıcı kayıt olabilir ve giriş yapabilir.
2. Giriş yapan kullanıcı kategori oluşturabilir.
3. Kullanıcı yeni gelir ve gider kaydı ekleyebilir.
4. İşlem kayıtları düzenlenebilir ve silinebilir.
5. İşlemler tarih ve kategoriye göre filtrelenebilir.
6. Sayfalama doğru toplam kayıt ve sayfa bilgisiyle çalışır.
7. Toplam gelir ve toplam gider doğru hesaplanır.
8. Bakiye, gelir eksi gider sonucuyla eşleşir.
9. Aylık grafik işlem kayıtlarıyla aynı toplamaları gösterir.
10. Bir kullanıcı başka kullanıcının verilerine erişemez.
11. Access token yenileme ve çıkış işlemleri doğru çalışır.
12. Sayfa yenilendiğinde SQL Server'daki kayıtlar kaybolmaz.
13. API işlemleri Swagger veya Postman üzerinden test edilebilir.

13. Gereksinim izlenebilirliği

Gelir-gider CRUD API ve arayüz entegrasyon testleri
Kategori yönetimi API ve arayüz entegrasyon testleri
Tarih/kategori filtresi Filtre sorgusu testleri
Gelir, gider ve bakiye Bilinen örnek veriyle hesaplama testi
Form doğrulamaları Geçersiz giriş senaryoları
SQL Server kalıcılığı Uygulama yeniden başlatma testi
Kullanıcı ve JWT Kimlik doğrulama ve yetkilendirme testleri
Aylık grafik API toplamı ile grafik karşılaştırması
Sayfalama Farklı sayfa ve boyut senaryoları

14. Varsayımlar ve riskler

Varsayımlar

- İlk sürüm tek para birimiyle çalışacaktır; arayüzde Türk lirası kullanılacaktır.
- Uygulama internet tarayıcısından kullanılacaktır.
- Kullanıcı işlemleri kendi yerel tarihini girer, sistem oluşturma zamanlarını UTC saklar.
- Silinen işlem kaydı için ayrı bir denetim geçmişi tutulmayacaktır.

Riskler ve önlemler

Kullanıcı verilerinin karışması Her sorguda JWT'den alınan kullanıcı kimliğini filtre olarak kullanmak

Para hesaplama hataları `decimal(18,2)` kullanmak ve örnek toplam testleri yazmak

Kategori ile işlem türünün uyuşmaması API ve veritabanı ilişkisinde tür-kullanıcı uyumunu doğrulamak

Token hırsızlığı Kısa ömürlü access token, hashlenmiş refresh token ve token rotasyonu kullanmak

Büyük işlem listesinde yavaşlama Veritabanı seviyesinde sayfalama ve indeks kullanmak

Gizli bilgilerin Git'e eklenmesi User Secrets veya ortam değişkenleri kullanmak ve `.gitignore` kontrolü yapmak `

15. Geliştirme sırası

- 1-2. gün: Analiz, gereksinim raporu ve veritabanı tasarımı.
- 3-5. gün: Web API, entity, `DbContext` , migration ve SQL Server kurulumu.
- 6-13. gün: Kategori, işlem, filtre, dashboard, doğrulama ve hata yönetimi.
- 14-18. gün: Angular ekranları, formlar, kategori yönetimi, özet ve filtreler.
- 19-20. gün: Test, hata düzeltme, kod temizliği, README ve sunum.

Opsiyonel kimlik doğrulama, grafik ve sayfalama özellikleri zorunlu akışları geciktirmeyecek şekilde ilgili backend ve frontend günlerine dağıtılacaktır.